Iterative deepening search

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def dls(node, goal, limit, path):

    if node == goal:
        return path + [node]

    if limit == 0:
        return None

    for child in graph[node]:
        if child not in path:
            result = dls(child, goal, limit - 1, path + [node])

            if result:
                return result

    return None


def ids(start, goal, max_depth):
```

```
    for depth in range(max_depth + 1):

        print("Searching at depth:", depth)

        result = dls(start, goal, depth, [])

        if result:
            return result

    return None


# Input
start = input("Enter initial state: ")
goal = input("Enter goal state: ")
max_depth = int(input("Enter maximum depth: "))

# Search
result = ids(start, goal, max_depth)

if result:
    print("Goal Found")
    print("Solution Path:", " -> ".join(result))
    print("Solution Depth:", len(result) - 1)
else:
    print("Goal not found")
```

Output:

```
PS C:\Users\n6787\OneDrive\Desktop\c> python -u "c:\Users\n6787\OneDriv
Enter initial state: A
Enter goal state: B
Enter maximum depth: 3
Searching at depth: 0
Searching at depth: 1
Goal Found
Solution Path: A -> B
Solution Depth: 1
PS C:\Users\n6787\OneDrive\Desktop\c> python -u "c:\Users\n6787\OneDriv
Enter initial state: A
Enter goal state: G
Enter maximum depth: 3
Searching at depth: 0
Searching at depth: 1
Searching at depth: 2
Goal Found
Solution Path: A -> C -> G
Solution Depth: 2
PS C:\Users\n6787\OneDrive\Desktop\c> python -u "c:\Users\n6787\OneDriv
Enter initial state: A
Enter goal state: Z
Enter maximum depth: 2
Searching at depth: 0
Searching at depth: 1
Searching at depth: 2
Goal not found
PS C:\Users\n6787\OneDrive\Desktop\c> python -u "c:\Users\n6787\OneDriv
Enter initial state: A
Enter goal state: G
Enter maximum depth: 2
Searching at depth: 0
Searching at depth: 1
Searching at depth: 2
Goal Found
Solution Path: A -> C -> G
Solution Depth: 2
```